Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Deprem sonrası online başlayan dönemin işleyişi ve ders düzeni.
- Değerlendirme ölçütleri, kopya kuralları, ödev teslim ve puanlama kuralları.
- Kaynak kitaplar ve 14 haftalık ders planı.
- Algoritma kelimesinin kökeni: El-Harezmi.
- Algoritma ve algoritma tasarımı kavramları, günlük hayattan örnekler.
- Bir algoritmanın taşıması gereken özellikler.
- Genel algoritmik problemler: sıralama (sorting), arama (searching), metin işleme, graf problemleri, kombinatoryal, geometrik ve nümerik problemler.
- Veri, veri tipi ve verinin bellekte saklanması.
- Veri yapısı (data structure) kavramı ve neden gerektiği; seyrek matris (sparse matrix) örneği.
- Algoritma hazırlama süreci: tasarım, ispat, analiz, gerçekleştirme ve test.
- Sözde kod (pseudo code) gösterimi.
- Akış diyagramı (flowchart) bileşenleri ve kuralları.
- Algoritma geliştirme ve iyileştirme; asal sayı bulma ve Eratosthenes Kalburu (Sieve of Eratosthenes).

Hocanın Özellikle Vurguladığı Kısımlar

- Değerlendirme ölçütleri dönem boyunca değişebilir. Hoca bunu birçok kez tekrarladı.
- Dersler online olduğu için kısa ve yoğun tutulacak: yaklaşık 40-45 dakikalık bölümler ve 10 dakikalık aralar.
- Kopya: kodlar çapraz taranıyor. Değişken adını, döngü yapısını değiştirmek ya da kontrolleri tersten yazmak benzerliği gizlemiyor.
- Bütün ödev, proje ve sınavlarda yalnız ANSI C kullanılır. C++ ve C# kabul edilmez.
- Ödevde istenen işlevi hazır kütüphane fonksiyonuyla değil, kendiniz yazmalısınız. Örneğin alt dizgi arama ödevinde hazır fonksiyon çağrılmaz.
- Ödevi son dakikaya bırakmayın. En az 10-15 dakika önce yükleyin.
- Algoritma hiçbir programlama diline bağlı olmamalıdır. Akış diyagramında `a++` gibi dile özgü operatör kullanılmaz; `a = a + 1` yazılır.
- Akış diyagramında tek başlangıç ve tek bitiş olmalı.
- `if` kutusunda true kolu boş, false kolu dolu bırakılmaz. Gerekirse koşul ters yazılır.
- Döngüden `break` ile çıkılmaz. Adım sayısı belli değilse `while` kullanılır.
- `while` döngüsünün içinde koşulu değiştiren bir işlem olmalı, yoksa döngü sonsuza kadar döner.
- Diziyi okumadan önce boyutu okunmalıdır.
- Doğru veri yapısını seçmek, zaman ve bellek açısından çok büyük kazanç sağlayabilir.
- Tek bir “en iyi algoritma” yok. Seçim probleme, ortamın kısıtlarına ve beklentilere bağlıdır.

Kısa Tekrar Notları

- **Algoritma:** Bir problemi çözmek için arka arkaya uygulanan, kesin ve sonlu sayıda adımdan oluşan işlem kümesi. Doğru girdiler için doğru çıktıyı sonlu sürede üretmelidir.
- **Program:** Bir algoritmanın bir programlama dilinde ifade edilmiş hâli.

- **Algoritmanın özellikleri:**
  - Anlaşılırlık, belirsiz olmama (unambiguity).
  - İyi tanımlanmış sıfır ya da daha fazla girdi (input).
  - İyi tanımlanmış bir ya da daha fazla çıktı (output).
  - Sonluluk (finiteness).
  - Yapılabilirlik (feasibility).
  - Bağımsızlık (independence): programlama dilinden ve kütüphaneden bağımsız olma.
- **Veri:** Algoritmaların işlediği temel elemanlar. Sayı, metin, resim, ses; bellekte hepsi sonunda 0 ve 1'lerdir.
- **Veri yapısı:** Bilginin algoritmalarda kullanılmak üzere belirli bir düzende ve belirli işlemlerle saklandığı yapı. Örnekler: dizi, bağlı liste, yığın (stack), kuyruk (queue), hash tablosu, ağaç (tree), heap, öncelikli kuyruk (priority queue).
- **Algoritma hazırlama süreci:** tasarım (design), ispat/doğrulama (validation), analiz (analysis), gerçekleştirme (implementation), test.
- **Zaman ve yer karmaşıklığı:** Hızlı algoritma fazla bellek isteyebilir. Gömülü sistemde bellek, borsa sisteminde hız öne çıkar.
- **Eratosthenes Kalburu:** $\sqrt{n}$'e kadar her asal p için, p^2'den başlayarak p'nin katları silinir. Kalanlar asal sayılardır.

Detaylı Açıklamalar

Dersin işleyişi ve değerlendirme

- **Ders saatleri:** Teorik ders pazartesi 13.00-15.50. Uygulama salı 16.00-17.50.
- Nisan ayına kadar hiçbir derste uygulama ya da laboratuvar yapılmayacak. Bu yüzden ertesi gün (salı) ders yok.
- **Değerlendirme (değişebilir):**

Bileşen	Ağırlık
Vize sınavı (tek vize)	%35
3 ödev (her biri %5)	%15
Proje	%10
Final sınavı	%40

- Önceki dönemlerde iki vize ve dört ödev vardı. Online dönem için azaltıldı.
- Proje, dönemin son haftalarında verilecek ve daha büyük bir ödev gibi olacak.
- Sınavların online mı, yüz yüze mi olacağı belli değil. YÖK ve senato kararı uygulanacak.
- Yoklama alınmayacak. Devamsızlıktan kalma yok.
- Hiçbir sınava girmeyen ve hiçbir ödevi vermeyen öğrenci F0 alır.

Ödev kuralları ve puan kesintileri

- Sisteme yüklenen dosya değerlendirilir. E-postayla, WhatsApp'la ya da başka yolla gönderilen ödev kabul edilmez.
- İnternetteki bir kaynakla ya da başka bir ödevle %50'den fazla benzerlik kopya sayılır. Kopyada ödevden ya da sınavdan sıfır alınır; disiplin süreci de başlayabilir.
- Ödevinizi GitHub'da paylaşacaksanız değerlendirme bitene kadar herkese açık (public) yapmayın. Yoksa kopya gibi görünebilir.

Kural ihlali	Kesinti
Dev-C++ (GCC 4.9.2) ile derlenmeyen kod	15 puan
ANSI C standardına uymamak	5 puan
Global değişken kullanmak	5 puan
Büyük dizi ve matrisler için statik bellek kullanmak (dinamik ayırma gerekli)	10 puan
Dosya yolunu mutlak (absolute) vermek; göreli (relative) yol gerekli	5 puan

Kural ihlali	Kesinti
Okunamayan ekran görüntüsü (örneğin telefonla çekilmiş ekran)	yaklaşık 20 puan

- ANSI C'de değişkenler bloğun başında tanımlanır. C99'daki gibi ilk kullanıldığı yerde tanımlamak (örneğin `for (int i = 0; ...)`) geçersizdir.
- Dosya yolu örneği: `C:\Users\user\Desktop\folder\input.txt` yanlış, `input.txt` doğru.
- **Rapor:** Kapak sayfasında ad-soyad, ders adı, grup, ödev numarası ve e-posta olmalı. Bölümler: giriş (problem tanımı), yöntem (çözüm yaklaşımı), algoritma analizi, sonuçlar ve yorum.
- Akış diyagramı Draw.io ya da Visio ile çizilebilir. Elle çizilirse okunaklı olmalı.
- **Dosya adları:** `ogrencino.c` ve `ogrencino.pdf`. Büyük dosyalar zip ya da rar ile sıkıştırılabilir; Drive ya da Dropbox linki kabul edilmez.
- Sistemsel bir hata olursa ekran görüntüsü alıp hocaya ve UZEM'e başvurun.

Kaynak kitaplar

- Anany Levitin, *Introduction to the Design and Analysis of Algorithms* (2. ya da 3. baskı).
- Robert Sedgewick, *Algorithms* (Part 1 ve Part 2).
- Cormen ve diğerleri, *Introduction to Algorithms* (genellikle 3. baskı).
- Slaytlar genellikle birkaç hafta biriktikten sonra, örneğin sınavdan önce, toplu yayınlanacak.

Ders planı

Hafta	Konu
1	Giriş: algoritmik problem, veri, veri yapısı, akış diyagramı tekrarı
2	Algoritma analizi, Big O ve karmaşıklık
3	Özyinelemeli (recursive) algoritmalar ve analizi
4	Diziler, bağlı listeler (linked list)
5	Kuyruk (queue) ve yığın (stack)
6	Ağaçlar, ikili ağaç ve ikili arama ağacı (BST)
7	Heap ve heap sort
8	Öncelikli kuyruk (priority queue)
9	Vize (Ramazan Bayramı'ndan sonraki hafta)
10	1 Mayıs tatili, ders yok
11-12	Graflar ve minimum yayılan ağaç (minimum spanning tree)
13	Metinde (string) arama
14	Karşılaştırma yapmayan sıralama algoritmaları

Algoritmanın kökeni

- Ebu Cafer Muhammed bin Musa El-Harezmi (yaklaşık 780-850), Bağdat'ta yaşamış bir matematikçi ve gökbilimci.
- Hint rakamlarıyla hesap yapmayı anlatan eseri Latinceye *Algoritmi de numero Indorum* adıyla çevrildi. "Algoritma" kelimesi buradan gelir.

Algoritma ve algoritmik problemler

- İşlemsel çözüm gerektiren bir problemi çözmek için izlenecek yolu kurmaya algoritma tasarlamak denir.
- Örnekler: bir sayı dizisini sıralamak, navigasyonda en kısa yolu bulmak, zaman planı yapmak, web aramasında sonuçları sıralamak.

- Diş fırçalama örneği: fırçayı al, macun sür, fırçala, durula, yerine koy, dur. Kesin ve sonlu adımlar, belirli girdiler ve çıktılar var.
- **Neden bir algoritmayı diğerine tercih ederiz?**
  - Hız (çalışma zamanı).
  - Bellek (yer). Örneğin 86 milyon TC kimlik numarasını sıralarken hızlı bir algoritma 86 milyonluk ek bellek isteyebilir. Bu bellek yoksa algoritma uygulanamaz (feasible değildir).
- **Arama:** Dizide bir eleman aranırken bulunduğu anda durmak daha iyidir. Bu, `for` + `break` ile değil, "sona gelene ya da bulana kadar" koşullu bir `while` ile yapılmalıdır.
- Girdi sıralıysa bu özellik kullanılır: ikili arama (binary search) çok daha az adımda sonuç verir.

Veri ve bellekte saklanması

- Bellekte her şey 0 ve 1'dir. Okumayı kolaylaştırmak için bitler 4'lü gruplanıp onaltılık (hexadecimal) basamaklarla yazılır.
- Bir kod tablosuyla bu sayılar karaktere çevrilir: `0x42` = `B`, `0x41` = `A` (onluk tabanda 65). `42 41 42 41` dizisi metin olarak "BABA" okunur.
- İşlemcide yazmaçlar (register) ve bir sonraki komutun adresini tutan program sayacı (program counter) vardır. İşlemci bellekteki makine kodlarını adım adım çalıştırır.

Veri yapısı neden gerekli: seyrek matris örneği

- 1000×1000'lik bir matris 1 milyon göz ister.
- Matriste yalnız ilk sütun, son sütun ve köşegen dolu, gerisi 0 olsun. Dolu göz sayısı yaklaşık 3000'dir.
- Yalnız dolu gözleri (satır, sütun, değer) üçlüsü olarak saklayan seyrek matris (sparse matrix) yapısı yaklaşık 9000 değer tutar.
- Kazanç yaklaşık 100 kat. Doğru veri yapısı hem zaman hem bellek kazandırır.

Sözde kod

- Yarı programlama dili, yarı konuşma dili gibi bir gösterim. İnternette algoritmalar çoğunlukla sözde kodla verilir. Bu derste çok kullanılmayacak.
- Kurallar:
  - Algoritmanın adı, girdileri ve çıktıları belirtilir.
  - Açılan her blok `end` ile kapatılır (`end if`, `end while`, `end for`).
  - Değişkenler tanımlanmaz, ilk kullanıldığı yerde tanımlanmış sayılır. Kullanılan veri yapısı ise mutlaka belirtilir.
  - Atama `:=` ya da ← ile gösterilir. Eşitlik kontrolü tek `=` ile yapılır.
  - Adımlar 1, 2, 2.1, 3 gibi numaralanabilir.
  - Değişken adları anlamlı olmalı: `x`, `y` yerine `notOrtalamasi` gibi.
- Dizinin toplamı örneği:

```
Algoritma DiziToplami
  Girdi: n elemanlı tam sayı dizisi
  Çıktı: dizinin elemanlarının toplamı
  1. toplam ← 0
  2. for i ← 1 to n do
     2.1 toplam ← toplam + dizi[i]
  3. end for
  4. return toplam
end DiziToplami
```

Akış diyagramı kuralları

- **Başla/Bitir:** Oval (elips) ile gösterilir. Tek başlangıç ve tek bitiş olmalı. Her daldan ayrı bitişe gitmek yanlıştır.
- **Girdi okuma:** Delikli kart (punch card) sembolü kullanılır. Okunan değişkenler virgülle ayrılır.

  - Toplam ya da ortalama gibi iç değişkenler okunmaz.
  - Dizi, `for` döngüsü içinde tek tek ya da kısa gösterimle (`A[n]`) okunabilir. Önce boyut (`n`) okunmalıdır.
- **Bağlantılar:** Oklarla gösterilir. Köşeli ve düzgün çizilmeli. Karışık akışlarda numaralı bağlantı noktaları kullanılabilir.
- **İşlem kutusu:** Dikdörtgen. Atama ya da aritmetik işlem yazılır, dile özgü operatör kullanılmaz.
- **Karar (`if`):** Altıgen ile gösterilir; eski kaynaklarda eşkenar dörtgen (baklava) görülür.
  - True (T) ve false (F) kolları mutlaka etiketlenir.
  - Tüm `if`'lerde T kolları aynı tarafta olmalı.
  - True kolu boş bırakılmaz. Örneğin "A = B ise hiçbir şey yapma, değilse A'yı artır" yerine koşul "A ≠ B" yapılır.
- **`for` döngüsü:** Üç bölmeli dikdörtgen: ilk değer, bitiş değeri, artım miktarı. Döngü gövdesi sağ tarafa çizilir, döngü bitince akış alttan devam eder.
- **`while` döngüsü:** Adım sayısı belli değilse kullanılır. Gövdede koşulu değiştiren bir işlem olmalı.
- **`do-while`:** Gövde önce çalışır, koşul sonda kontrol edilir.

Dizi toplamı örneğinin izlenmesi ($n = 3$, dizi = 3, 5, 8):

Adım	`i`	`toplam`
Başlangıç	-	0
1	1	3
2	2	8
3	3	16
Çıkış	4	16 (ekrana yazılır)

Algoritma geliştirme ve iyileştirme

- Önce problemi doğru çözen basit bir algoritma tasarlanır. Bu, performans karşılaştırması için temel çizgi (baseline) olur.
- Sonra algoritma iyileştirilir. Veri yapısı ve algoritma birlikte düşünülmelidir.
- Bir işi 10 kat hızlandırmak için 10 kat güçlü bilgisayar almak yerine algoritmayı iyileştirmek çoğu zaman daha akıllıcadır.

Asal sayıları bulma ve Eratosthenes Kalburu

- **Basit yöntem:** Her sayıyı kendisinden küçük sayılara bölmek. İç içe iki döngü, yaklaşık n^2 işlem.
- **İyileştirme:** Bölenleri $\sqrt{n}$'e kadar denemek yeterlidir.
- **Eratosthenes Kalburu (1'den 20'ye kadar örnek):**
  - 2 kalır, 2'nin katları silinir.
  - 3 kalır, 3'ün katları silinir.
  - 5 ve 7 için katları zaten silinmiş durumdadır.
  - $\sqrt{20}$'den sonrasına bakmaya gerek yok.
  - Sonuç: 2, 3, 5, 7, 11, 13, 17, 19.
- Slayttaki sözde kod:

```
Input: Integer n >= 2
Output: n'den küçük eşit asal sayılar

for p <- 2 to n do A[p] <- p

for p <- 2 to floor(sqrt(n)) do
    if A[p] != 0              // p önceden listeden silinmediyse
        j <- p * p
        while j <= n do
            A[j] <- 0         // asal değil olarak işaretle
            j <- j + p
```

- Ödev niteliğinde: Bu algoritmanın akış diyagramını kendiniz çizin. Gelecek hafta iki farklı akışı incelenecek.

Duyurular

- OBS'de gruplar dengesiz dağıtılmış. Bu hafta gruplar yeniden dağıtılacak; bazı öğrenciler başka hocayla devam edecek.
- OBS üzerinden "online derslere katılım formu" gönderildi. İnternet erişimi ve cihaz durumunu soran kısa bir anket; doldurulması isteniyor.
- Derse 42 öğrenci katıldı; grupta yaklaşık 90 öğrenci var.

Ders sonrası notlar

- Hoca 4 bitlik gruplamaya "binary coded decimal" dedi. Yaptığı şey onaltılık (hexadecimal) gösterimdir. BCD, her onluk basamağı ayrı 4 bitle kodlar ve yalnız 0-9 değerlerini kullanır.
- Hoca *Algoritmi de numero Indorum* çevirisini "Algebra" kitabına bağladı ve çevirinin 1300'lerde yapıldığını söyledi. Bu Latince çeviri, büyük olasılıkla El-Harezmi'nin Hint rakamlarıyla hesap üzerine yazdığı aritmetik kitabına aittir ve 12. yüzyılda yapılmıştır. Cebir kitabı ayrı bir eserdir: *el-Kitâb el-Muhtasar fî Hisâb el-Cebr ve'l-Mukâbele*.
- Seyrek matris örneğinde dolu göz sayısı için "yaklaşık 3500" dendi. İlk sütun, son sütun ve köşegen için doğru sayı $3 \cdot 1000 - 2 = 2998$'dir; (satır, sütun, değer) olarak yaklaşık 9000 değer tutulur. Hocanın vardığı "yaklaşık 100 kat daha az yer" sonucu yine geçerli.
- $\sqrt{n}$ kuralının gerekçesi: n bileşik sayıysa $n = a \cdot b$ olur ve a ile b'den en az biri $\sqrt{n}$'den küçük ya da eşittir. Yani $\sqrt{n}$'e kadar böleni yoksa n asaldır. Derste söylenen "sonrakilerin hepsi öncekilerin çarpımı" ifadesi bu kuralın tam karşılığı değil.
- $\sqrt{20} \approx 4{,}47$ olduğu için kalbur yalnız 2 ve 3 için çalıştırılmalıdır. Derste 5 ve 7'ye de bakıldı; bu adımlar sonucu değiştirmez ama gereksizdir.
- Eratosthenes Kalburu'nun karmaşıklığı $O(n \log \log n)$'dir.
- Algoritma özelliklerinden "fitness" diye geçen terimin doğrusu sonluluk (finiteness)'tur.
- Altyazıda "Hidmet H" diye geçen kütüphane `math.h`'tir. Rapor uzantısı `.pdf`'dir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.